

Frontend · Mundo Pokémon — Fase 2 · Continuidad: interacciones, transiciones y animaciones

Continuación de una práctica obligatoria

Esta hoja continúa la práctica **Frontend · Mundo Pokémon — Fase 2 · Maquetación CSS**.

Debes trabajar sobre la versión que ya tienes maquetada.

Propósito de esta continuidad

En esta continuación no vas a rehacer la maquetación principal. Vas a **enriquecerla**.

El objetivo es que la interfaz deje de ser solo una composición estática y empiece a comunicar mejor al usuario:

- qué elementos parecen interactivos;
- qué tarjeta está bajo el cursor o bajo foco;
- cómo cambian ciertos elementos entre estados;
- y cómo puede usarse el movimiento de forma sobria para reforzar la experiencia visual.

Contexto dentro del proyecto

Esta práctica enlaza con lo trabajado en la sesión sobre:

- estados interactivos;
- transiciones;
- transformaciones;
- animaciones con `@keyframes` ;
- y uso moderado de efectos visuales.

La idea no es “decorar por decorar”, sino mejorar una interfaz ya construida con decisiones visuales más conscientes.

1. Competencias que vas a trabajar

Al terminar esta continuidad deberías ser capaz de:

- añadir estados visuales útiles a una interfaz ya maquetada;
 - aplicar transiciones con sentido entre distintos estados;
 - usar transformaciones de forma controlada;
 - incorporar animaciones sencillas cuando aporten claridad;
 - distinguir entre una mejora visual útil y un efecto innecesario;
 - y mantener la coherencia del CSS al ampliar un proyecto existente.
-

2. Organización del trabajo

2.1. Carpeta de trabajo

Continúa trabajando sobre la misma carpeta del proyecto:

```
frontend/  
├── mundo-pokemon/  
│   ├── index.html  
│   └── styles.css
```

2.2. Recomendación de rama

Si trabajas con ramas, puedes usar una rama específica para esta continuidad.

Ejemplo recomendado:

```
feature/frontend-mundo-pokemon-phase-02-interactions
```

Importante

No se trata de hacer una segunda versión separada de la interfaz.
Se trata de mejorar la que ya tienes.

3. Punto de partida

Antes de comenzar esta continuidad, tu proyecto debería tener ya resuelto, como mínimo:

- una composición general funcional;
- una caja de búsqueda visualmente clara;
- una secuencia o rejilla de tarjetas;
- una estructura interna consistente para cada tarjeta;
- una integración razonable de las imágenes;
- y una base responsive funcional.

Si tu fase 2 todavía no está razonablemente asentada, conviene reforzar primero esa parte antes de añadir interacciones y animaciones.

Error frecuente

No intentes tapar una maquetación débil con efectos visuales.

Las transiciones y animaciones mejoran una interfaz bien construida, pero no arreglan una estructura pobre ni una jerarquía mal resuelta.

4. Reglas generales de esta continuidad

En esta práctica:

- trabaja sobre tu `styles.css` actual;
- mantén el CSS ordenado por bloques lógicos;
- no conviertas la interfaz en una sucesión de efectos;
- usa transiciones y animaciones solo cuando aporten algo;
- no elimines el foco visible sin ofrecer una alternativa clara;
- y recuerda que el movimiento debe ayudar a entender la interfaz, no distraer.

Criterio general

Si dudas entre “quedará vistoso” y “quedará claro”, prioriza siempre **claridad**.

5. Tarea principal

Debes enriquecer tu interfaz de **Mundo Pokémon** incorporando **interacciones, transiciones y animaciones** de forma coherente con la maqueta ya construida.

5.1. Estados interactivos en tarjetas

Las tarjetas de Pokémon deben empezar a sugerir visualmente que son elementos importantes dentro de la interfaz y que, en fases posteriores, podrán tener comportamiento más rico.

Debes incorporar alguna mejora visual cuando la tarjeta:

- esté bajo el cursor;
- reciba foco si la estructura lo permite;
- o pase a un estado activo si has preparado algún elemento interactivo interno.

Qué puedes plantearte

Piensa en cosas como:

- cambios de sombra;
- ligeros cambios de color;
- pequeñas variaciones de escala;
- desplazamientos muy cortos;
- o refuerzos visuales del borde o contorno.

5.2. Transiciones

Debes aplicar al menos algunas transiciones donde tenga sentido.

No se espera que toda la interfaz transicione, sino que identifiques bien qué cambios merecen suavizarse.

Por ejemplo:

- cambio de fondo o color;
- cambio de sombra;
- pequeña transformación;
- cambios en botones, enlaces o tarjetas.

Lo importante aquí

No basta con poner `transition` en muchos sitios.

Debes pensar:

- qué cambia;
- por qué cambia;
- y si realmente merece una transición.

5.3. Transformaciones

Debes usar al menos una transformación con sentido dentro de la interfaz.

Puede ser algo relacionado con:

- una tarjeta;
- una imagen;
- un botón;
- un icono;
- o un elemento de apoyo visual.

La transformación debe ser **moderada** y coherente con el conjunto.

5.4. Animación sencilla con `@keyframes`

Debes incorporar al menos una animación sencilla mediante `@keyframes`.

Puede aplicarse, por ejemplo, a:

- la entrada inicial de algún bloque;
- la aparición de tarjetas;
- un aviso o elemento puntual;
- o algún recurso visual secundario que no rompa la lectura de la interfaz.

Evita esto

No conviertas todas las tarjetas en una secuencia exagerada de animaciones largas o llamativas.

La práctica no busca espectáculo, sino criterio.

6. Qué debes revisar al terminar

Antes de dar esta continuidad por buena, revisa si:

- la interfaz comunica mejor qué elementos son relevantes o interactivos;
 - las transiciones son suaves pero no lentas;
 - las transformaciones no rompen la composición;
 - las animaciones no distraen;
 - la interfaz sigue siendo clara aunque elimines mentalmente los efectos;
 - y el CSS ha crecido con orden, no con parches.
-

7. Orientaciones de trabajo

Cómo abordarlo

Una estrategia razonable puede ser esta:

1. revisa primero qué elementos de tu interfaz ya podrían parecer interactivos;
2. decide después qué estado visual quieres reforzar;
3. añade transiciones solo en esos cambios;
4. incorpora una transformación breve y contenida;
5. añade por último una animación sencilla con `@keyframes` ;
6. y revisa si el conjunto sigue siendo limpio.

Organización del CSS

Si vas a ampliar tu archivo de estilos, conviene que agrupes claramente la nueva parte.
Por ejemplo:

- base global;
- layout;
- componentes;
- estados interactivos;
- animaciones.

No es obligatorio seguir exactamente ese orden, pero sí mantener una lógica clara.

8. Criterios de calidad

Esta continuidad se considerará bien resuelta si:

- las interacciones ayudan a entender mejor la interfaz;
- las tarjetas responden visualmente de forma clara y sobria;
- las transiciones están bien elegidas;
- existe al menos una transformación con sentido;
- existe al menos una animación sencilla con `@keyframes` ;
- los efectos no recargan la interfaz;
- la coherencia visual general se mantiene;
- y el CSS sigue siendo legible y ordenado.

Lo que se espera aquí

No se espera una interfaz “llena de movimiento”.

Se espera una interfaz mejor comunicada, más cuidada y más consciente de sus estados visuales.

9. Opcional · Llevar la maqueta a Tailwind CSS v4

Ampliación opcional

Si quieres ir un paso más allá, puedes intentar **reconstruir esta maqueta usando Tailwind CSS v4**.

Esta ampliación no sustituye la práctica principal. Debes hacerla solo si tu versión en CSS ya está bien asentada.

Qué se busca con esta ampliación

La idea no es “pasarle todo a clases utility porque sí”, sino comprobar si eres capaz de trasladar las mismas decisiones de interfaz a otro enfoque de trabajo.

Ahí deberías pensar en cómo reaparecen en Tailwind cosas como:

- espaciados;
- colores;
- tipografía;
- layout;

- estados `hover` o `focus` ;
- transiciones;
- transformaciones;
- y pequeñas animaciones o variantes visuales.

Criterio para esta ampliación

Si haces esta parte opcional, el objetivo no es que quede perfecto, sino que entiendas mejor la relación entre CSS nativo y un enfoque utility-first.

10. Cierre

Al terminar esta continuidad deberías haber dado un paso importante: convertir una interfaz ya maquetada en una interfaz que también **responde visualmente** con más claridad a la interacción.

Idea final

Una interfaz mejor no siempre es la que tiene más efectos.
Muchas veces es la que usa justo los necesarios para comunicar mejor lo que ocurre.